Software Requirements Specification (SRS)

Ultra-Low Resource LLM Context Compression Engine

Version: 1.0

---

1. System Overview

A web application (Next.js 14) where a user submits raw text context, the system compresses it through a two-stage pipeline, and displays compression metrics plus a live accuracy-retention proof by comparing LLM answers on full vs. compressed context.

2. Tech Stack

Layer	Technology
Frontend	Next.js 14 (App Router), React, Tailwind CSS
Backend	Next.js API routes (Node.js)
AI	Claude API (Anthropic) for semantic compression + Q&A validation
Database	Supabase (PostgreSQL) — for session/chunk/eval storage
Tokenizer	tiktoken or a simple approximate token counter
Deployment	Vercel

3. Functional Requirements

FR-1: Context Input

- FR-1.1: User can paste raw text into a textarea (max ~50,000 characters for MVP)
- FR-1.2: User can upload a `.txt`, `.log`, `.md`, or `.js/.py/.ts` code file
- FR-1.3: System extracts and displays raw text content and original token count

FR-2: Stage 1 — Structural Compression

- FR-2.1: System removes exact duplicate lines
- FR-2.2: System removes excess whitespace/blank lines
- FR-2.3: System detects and collapses near-duplicate sentences/lines (similarity threshold-based)
- FR-2.4: System strips common boilerplate patterns (e.g., repeated log timestamps/headers, repeated import blocks)
- FR-2.5: Output: intermediate compressed text + token count after Stage 1

FR-3: Stage 2 — Semantic Compression

- FR-3.1: System splits Stage-1 output into logical chunks (by paragraph, function, or log block)
- FR-3.2: System scores each chunk for information density/relevance (via Claude API call, batched)
- FR-3.3: System compresses/summarizes low-score chunks while preserving high-score chunks verbatim (entities, numbers, code logic, key facts must be retained)
- FR-3.4: Output: final compressed text + token count + total compression ratio

FR-4: Accuracy Validation

- FR-4.1: User provides (or system auto-generates) a test question relevant to the context
- FR-4.2: System sends the question + FULL original context to Claude API → Answer A
- FR-4.3: System sends the question + COMPRESSED context to Claude API → Answer B
- FR-4.4: System computes a similarity score between Answer A and Answer B (semantic similarity via embeddings, or a Claude-based judge call)
- FR-4.5: System displays both answers side-by-side with the similarity/retention score

FR-5: Metrics Dashboard

- FR-5.1: Display original token count, compressed token count, compression ratio (%)
- FR-5.2: Display estimated cost savings (based on current Claude API pricing per token)
- FR-5.3: Display latency comparison (time to get Answer A vs Answer B)
- FR-5.4: Display accuracy retention score (%)

FR-6: Diff View

- FR-6.1: System shows a visual diff (removed/compressed sections highlighted) between original and compressed text

FR-7: Session Logging

- FR-7.1: Each compression run is stored with: input text hash, original/compressed token counts, compression ratio, accuracy score, timestamp
- FR-7.2: User can view past runs in a simple history list (session-based, no auth required for MVP)

4. Non-Functional Requirements

Category	Requirement
Performance	Compression pipeline should complete within 15–20 seconds for a 5,000-token input
Usability	Single-page flow, no login required, results visible without scrolling excessively
Reliability	Graceful fallback to Stage-1-only compression if Claude API fails/times out
Scalability	Not a priority for MVP — single-user demo scale is sufficient
Security	No user auth/PII for MVP; API keys stored server-side only, never exposed to client
Cost control	Cap chunk-scoring calls (batch chunks, don't call Claude once per line)

5. System Architecture (High Level)

```
[Frontend: Next.js UI]
        |
        v
[API Route: /api/compress]
        |
        ├── Stage 1: Structural Compressor (pure JS/TS, no external call)
        |
        ├── Stage 2: Semantic Compressor
        |         ├── Chunker
        |         ├── Chunk Scorer (Claude API batch call)
```

```
        |          └── Chunk Compressor (Claude API call on low-score chunks)
        |
        v
[API Route: /api/validate]
        ├── Full-context Q&A (Claude API)
        ├── Compressed-context Q&A (Claude API)
        └── Similarity Scorer (embeddings or Claude judge call)
        |
        v
[Database: Supabase]
        └── Store run metadata, chunk scores, eval results
        |
        v
[Frontend: Results Dashboard]
```

6. Data Flow

1. User input → Frontend → POST `/api/compress`
2. Backend runs Stage 1 → Stage 2 → returns compressed text + metrics
3. Frontend sends test question → POST `/api/validate`
4. Backend runs both Q&A calls + similarity scoring → returns comparison
5. Backend writes run summary to Supabase
6. Frontend renders dashboard from combined API responses

7. API Contracts (Summary — full detail in Backend doc)

- `POST /api/compress` → `{ originalText, options }` → `{ compressedText, stage1Text, originalTokens, stage1Tokens, finalTokens, compressionRatio }`
- `POST /api/validate` → `{ originalText, compressedText, question }` → `{ answerFull, answerCompressed, similarityScore, latencyFull, latencyCompressed }`
- `GET /api/history` → `{ runs: [...] }`

8. Constraints

- 24-hour build window
- Must use Claude API (no other LLM providers for MVP)

- No multi-modal input handling required